

國立陽明交通大學
資訊科學與工程研究所
碩士論文

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University
Master's Thesis

Compile Once, Execute Many：重複性模板實例化任務
的可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification
Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞 (Lin, Kuan-Cheng)
指導教授： 陳添福 (Chen, Tien-Fu)

中華民國 一一五年十一月
November 2026

Compile Once, Execute Many：重複性模板實例化任務的
可驗證規格編譯方法

Compile Once, Execute Many: Verifiable Specification
Compilation for Recurrent Template-Instantiation Tasks

研 究 生： 林冠丞

Student： Kuan-Cheng Lin

指導教授： 陳添福 博士

Advisor： Dr. Tien-Fu Chen



November 2026
Taiwan, Republic of China

中華民國 一一五年十一月

誌 謝

(致謝待撰寫。)

林冠丞 於

國立陽明交通大學 資訊科學與工程研究所

中華民國 一一五年十一月



Compile Once, Execute Many：重複性模板實例化任務的可驗證規格編譯方法

學生：林冠丞

指導教授：陳添福 博士

國立陽明交通大學
資訊科學與工程研究所

摘 要

(中文摘要待撰寫。)

關鍵字：(待定，5-7 個)



Compile Once, Execute Many: Verifiable Specification Compilation for Recurrent Template-Instantiation Tasks

Student : Kuan-Cheng Lin

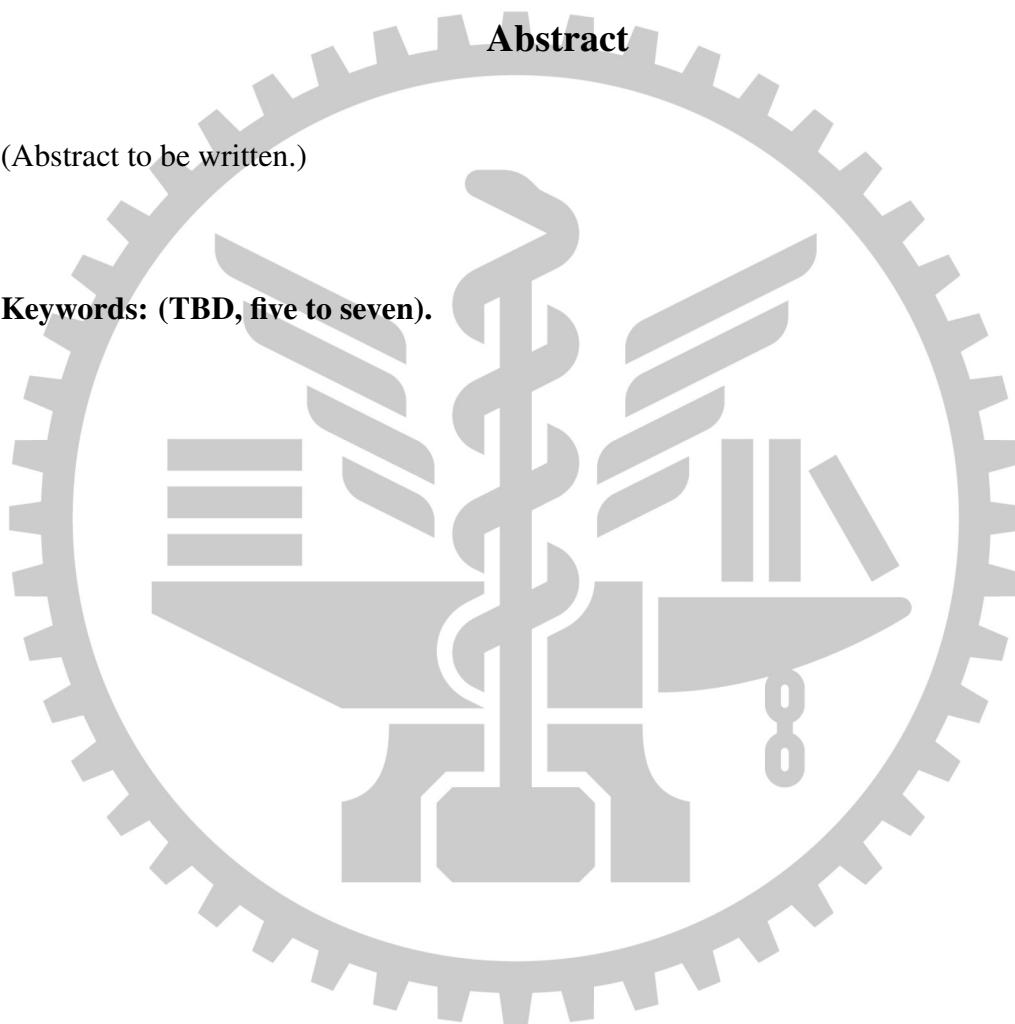
Advisor: Dr. Tien-Fu Chen

Institute of Computer Science and Engineering
National Yang Ming Chiao Tung University

Abstract

(Abstract to be written.)

Keywords: (TBD, five to seven).



Contents

Chinese Abstract	i
English Abstract	ii
Contents	iii
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Motivation	1
1.2 Two Existing Approaches, Both Wrong	1
1.3 Our Position	1
1.4 Contributions	1
1.5 Claims and Where They Are Tested	1
1.6 Thesis Organization	1
2 Related Work	2
2.1 Template-Shaped Tasks	3
2.2 Document Formats and Representation Mismatch	3
2.3 Failure Modes	3
2.4 Governance Requirements	3
2.5 LLMs for Document and Engineering-Drawing Understanding	3
2.6 Document Automation, RPA, and Template Filling	3

2.7	Programming by Example and Program Synthesis	3
2.8	Tool-Making, Skill Libraries, and Amortized LLM Computation	3
2.9	Execution-Feedback Program Repair	3
2.10	Verification and Guardrails for LLM Output	3
2.11	Cost-Aware Agent Evaluation	3
3	System Model	4
3.1	Problem Formulation	5
3.1.1	Compile-Once, Execute-Many	5
3.1.2	What the Derivation Consumes	5
3.1.3	Required Properties of the Specification IR	5
3.1.4	Design Principle: Fail Loud, Not Silent	5
3.1.5	Scope and Applicability Conditions	5
3.2	Domain-Dependent and Domain-Independent Layers	5
3.3	Specification IR	5
3.4	Structured Dump as Grounding	5
3.5	Locator Mechanisms	5
3.5.1	Semantic Locators	5
3.5.2	Range Locators and Source-Side Aggregation	5
3.5.3	Failure Modes and What Each Mechanism Blocks	5
3.6	Target Reference and Write-Back	5
3.7	Deterministic Compiler and Zero-LLM Runtime	5
3.8	Lifecycle Management	5
3.8.1	Lineage and Versioning	5
3.8.2	Regression Suite	5

3.8.3	Drift Detection	5
3.8.4	Auditability	5
3.9	Expressiveness Boundary	5
4	Methodology	6
4.1	State Machine.....	6
4.2	Structural Gates.....	6
4.3	Two Complementary Oracles.....	6
4.4	Case-Repair Loop	6
4.5	Human in the Loop: Evidence Review	6
4.6	Coverage Is the Real Bottleneck.....	6
5	Performance Evaluation.....	7
5.1	Research Questions.....	8
5.2	Study Design.....	8
5.2.1	Why Not a Few-Shot Held-Out Split.....	8
5.2.2	Two-Level Split.....	8
5.2.3	Dataset Construction.....	8
5.3	Baselines and Fairness	8
5.4	Metrics	8
5.4.1	Asymmetric Rerun Budget	8
5.5	The Evaluator.....	8
5.6	Template Drift.....	8
5.7	Ablations.....	8
5.8	Results.....	8
5.8.1	Where the Randomness Went.....	8

5.9 Case Study: Engineering Drawings	8
5.10 Threats to Validity	8
6 Conclusions	9
6.1 Conclusion	9
6.2 Limitations and When This Method Does Not Apply	9
6.3 Future Work	9
References	10



List of Figures



List of Tables



Chapter 1. Introduction

1.1 Motivation

1.2 Two Existing Approaches, Both Wrong

1.3 Our Position

1.4 Contributions

1.5 Claims and Where They Are Tested

1.6 Thesis Organization



Chapter 2. Related Work

2.1 Template-Shaped Tasks

2.2 Document Formats and Representation Mismatch

2.3 Failure Modes

2.4 Governance Requirements

2.5 LLMs for Document and Engineering-Drawing Understanding

2.6 Document Automation, RPA, and Template Filling

2.7 Programming by Example and Program Synthesis

2.8 Tool-Making, Skill Libraries, and Amortized LLM Computation

2.9 Execution-Feedback Program Repair

2.10 Verification and Guardrails for LLM Output



Chapter 3. System Model

3.1 Problem Formulation

3.1.1 Compile-Once, Execute-Many

3.1.2 What the Derivation Consumes

3.1.3 Required Properties of the Specification IR

3.1.4 Design Principle: Fail Loud, Not Silent

3.1.5 Scope and Applicability Conditions

3.2 Domain-Dependent and Domain-Independent Layers

3.3 Specification IR

3.4 Structured Dump as Grounding

3.5 Locator Mechanisms

3.5.1 Semantic Locators

3.5.2 Range Locators and Source-Side Aggregation

3.5.3 Failure Modes and What Each Mechanism Blocks

Chapter 4. Methodology

4.1 State Machine

4.2 Structural Gates

4.3 Two Complementary Oracles

4.4 Case-Repair Loop

4.5 Human in the Loop: Evidence Review

4.6 Coverage Is the Real Bottleneck





Chapter 5. Performance Evaluation

5.1 Research Questions

5.2 Study Design

5.2.1 Why Not a Few-Shot Held-Out Split

5.2.2 Two-Level Split

5.2.3 Dataset Construction

5.3 Baselines and Fairness

5.4 Metrics

5.4.1 Asymmetric Rerun Budget

5.5 The Evaluator

5.6 Template Drift

5.7 Ablations

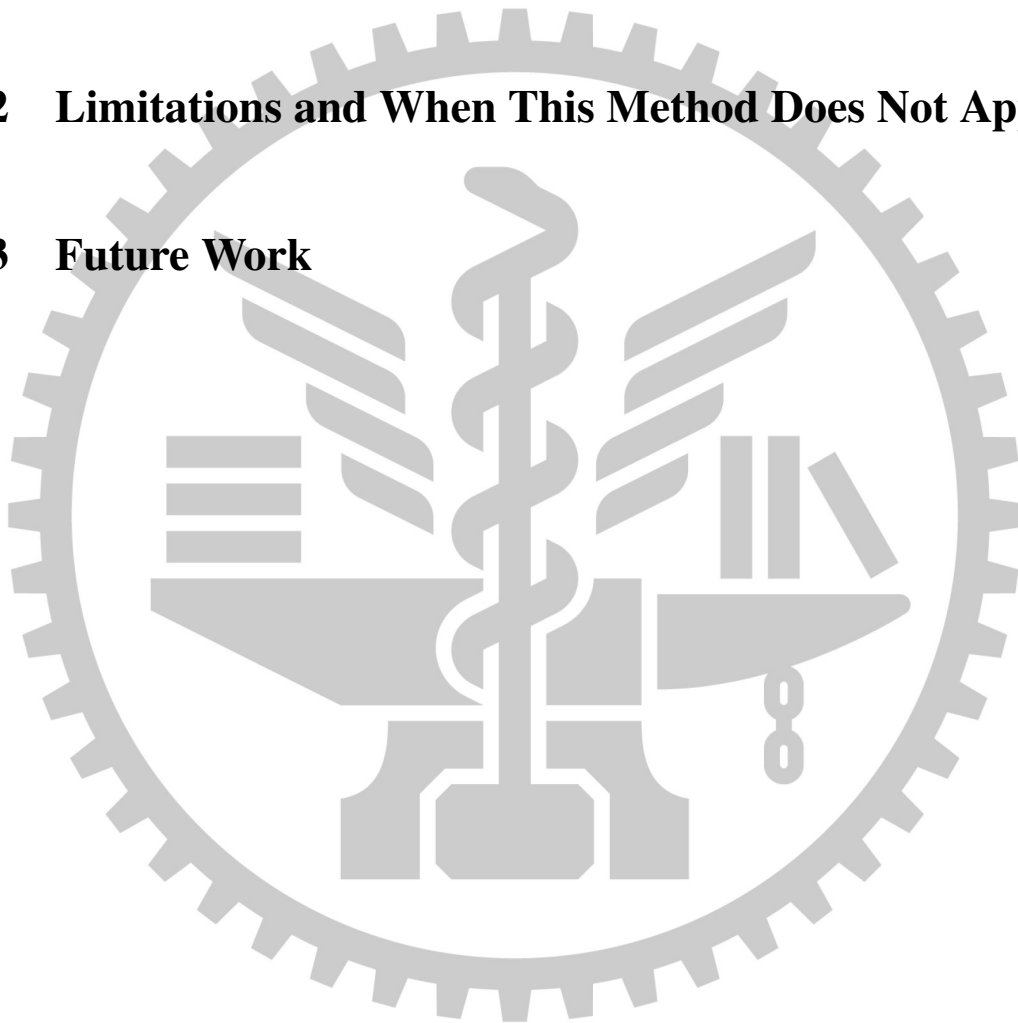
5.8 Results

Chapter 6. Conclusions

6.1 Conclusion

6.2 Limitations and When This Method Does Not Apply

6.3 Future Work



References

